

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1. How do serial and parallel blocks of code work together in a single program?

1 / 1 point

- ☐ When a parallel block of code is reached, the data resulting from the cores is continually being passed off to the serial code as the parallel code runs.
- ☒ When a parallel block of code is reached in a program, the code is distributed among the cores. The cores then complete the tasks, communicate as needed, and wait for all cores to finish before moving on to the serial block.
- ☐ A program must have only either serial or parallel blocks, but not both.
- ☐ All of the serial code in a program is run first, and then the parallel code is run.

👏 Nice work

2. Which is the best description of load imbalance?

1 / 1 point

- ☒ Processor 1 has a task that takes longer to run than Processor 2. Before the job can continue running past the current code block, Processor 2 must wait for Processor 1 to complete its task.
- ☐ Processor 1 and Processor 2 each run parts of the same task. Processor 1 and Processor 2 share the workload, and it generally isn't even.
- ☐ Processor 2 cannot start a task until Processor 1 finishes
- ☐ Processor 1 and Processor 2 have tasks that complete at the same time, but communication among other processors takes up time, causing non-uniformity.

👏 Nice work

When the tasks are distributed among the processors, the non-uniformity comes from one job requiring more resources, and thus taking longer, than the other.

3. How does speedup and efficiency help you determine how many resources you should use on an HPC system?

1 / 1 point

- ☒ Efficiency can tell you, for a fixed problem size, what a reasonable number of cores to use is.
- ☐ Can help you predict the amount of memory you will need with increased time to solution
- ☐ Will predict how granular your calculations should be in your program
- ☐ All of the above

👏 Nice work

An increase in core count with a fixed problem size decreases the time to solution. After you've added too many cores, there is too much overhead to be worth the extra resource cost

4. Which best describes a strong scaling scenario, when X and Y are used to describe your problem size, and N describes the number of cores?

1 / 1 point

- ☐ X and Y stay constant as N decreases, showing a decrease in the time to solution
- ☐ The values of X, Y, and N remain constant, and the time to solution stays fixed
- ☐ The values of X and Y increase while N increases, showing an increase in the time to solution.
- ☒ The values of X and Y stay constant as N increases, showing a decrease in the time to solution.

👏 Nice work

The purpose of strong scaling is to show that increasing the number of cores decreases your time to solution when you keep all other variables constant.

See video on scalability to review

5. Which best describes a weak scaling scenario, when X and Y are used to describe your problem, and N describes the number of cores?

1 / 1 point

- ☐ X and Y stay constant as N decreases, showing an increase in the work per core, and an increase in the time to solution.
- ☐ X and Y remain fixed as N increases. Work per core decreases, and time to solution decreases.
- ☒ X and Y increase as N increases. Work per core is constant, and time to solution is also constant for an ideal case.
- ☐ X, Y, and N remain constant, the work per core is constant, and time to solution is constant.

👏 Nice work

This is accurate. Here the purpose is to show that you can solve a larger problem, so showing an increase in work per core while time remains constant achieves this.

See video on scalability to review